
Lab Assignment 15

Name of Student : K. Chaitanya
Hallticket No. : 2303A51677
Batch No. 22

Task 1 – Student Attendance API

Create a RESTful API for student attendance tracking.

Instructions:**• Endpoints required:**

- o GET /attendance → View attendance records
 - o POST /attendance → Mark attendance
 - o PUT /attendance/{id} → Update attendance record
 - o DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling.**

Expected Output:

- API for maintaining and updating attendance records.**

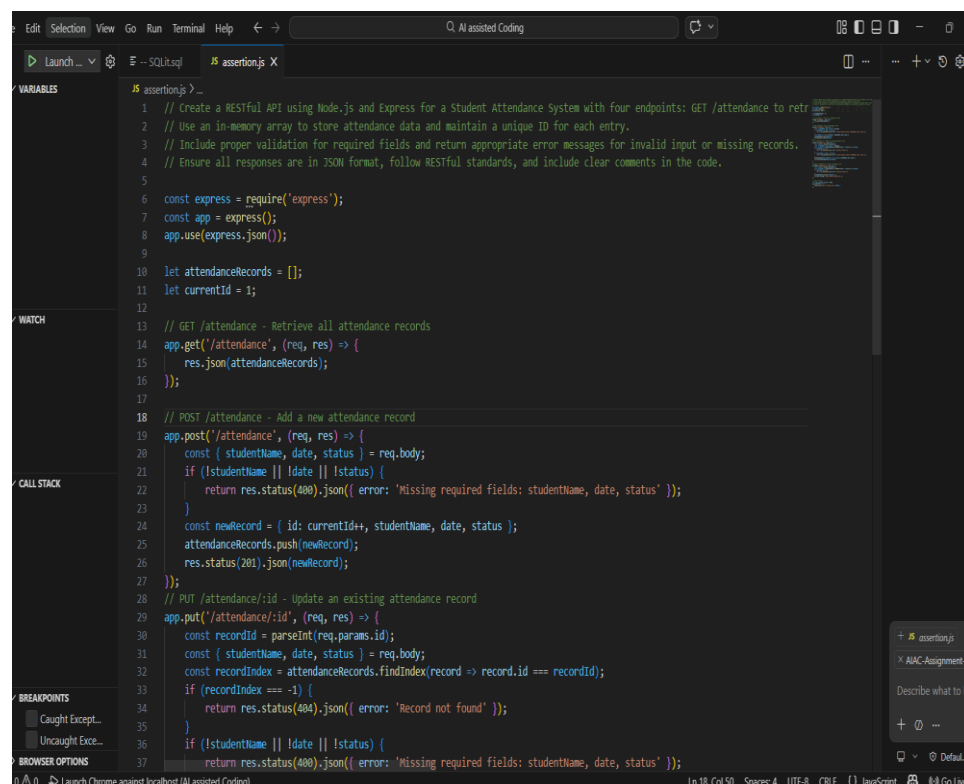
Prompt:-

Build a RESTful Student Attendance API using Node.js and Express with endpoints: GET /attendance, POST /attendance, PUT /attendance/:id, and DELETE /attendance/:id.

Store data in an in-memory array and assign a unique ID to each record.

Implement input validation and basic error handling for missing fields and invalid IDs.

Return all responses in JSON format with proper HTTP status codes and include clear comments.

Code:-

```
1 // Create a RESTful API using Node.js and Express for a Student Attendance System with four endpoints: GET /attendance to retr
2 // Use an in-memory array to store attendance data and maintain a unique ID for each entry.
3 // Include proper validation for required fields and return appropriate error messages for invalid input or missing records.
4 // Ensure all responses are in JSON format, follow RESTful standards, and include clear comments in the code.
5
6 const express = require('express');
7 const app = express();
8 app.use(express.json());
9
10 let attendanceRecords = [];
11 let currentId = 1;
12
13 // GET /attendance - Retrieve all attendance records
14 app.get('/attendance', (req, res) => {
15   res.json(attendanceRecords);
16 });
17
18 // POST /attendance - Add a new attendance record
19 app.post('/attendance', (req, res) => {
20   const { studentName, date, status } = req.body;
21   if (!studentName || !date || !status) {
22     return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
23   }
24   const newRecord = { id: currentId++, studentName, date, status };
25   attendanceRecords.push(newRecord);
26   res.status(201).json(newRecord);
27 });
28
29 // PUT /attendance/:id - Update an existing attendance record
30 app.put('/attendance/:id', (req, res) => {
31   const recordId = parseInt(req.params.id);
32   const { studentName, date, status } = req.body;
33   const recordIndex = attendanceRecords.findIndex(record => record.id === recordId);
34   if (recordIndex === -1) {
35     return res.status(404).json({ error: 'Record not found' });
36   }
37   if (!studentName || !date || !status) {
38     return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
39   }
40   attendanceRecords[recordIndex] = { id: recordId, studentName, date, status };
41   res.status(200).json(attendanceRecords[recordIndex]);
42 });
43
44 // DELETE /attendance/:id - Remove an attendance record
45 app.delete('/attendance/:id', (req, res) => {
46   const recordId = parseInt(req.params.id);
47   const recordIndex = attendanceRecords.findIndex(record => record.id === recordId);
48   if (recordIndex === -1) {
49     return res.status(404).json({ error: 'Record not found' });
50   }
51   attendanceRecords.splice(recordIndex, 1);
52   res.status(200).json({ message: 'Record removed successfully' });
53 });
54
55 const PORT = 3000;
56 app.listen(PORT, () => {
57   console.log(`Server is running on port ${PORT}`);
58 });
```

```

29 JS assertion.js > ...
30 app.put('/attendance/:id', (req, res) => {
31   return res.status(404).json({ error: 'Record not found' });
32 }
33 if (!studentName || !date || !status) {
34   return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
35 }
36 attendanceRecords[recordIndex] = { id: recordId, studentName, date, status };
37 res.json(attendanceRecords[recordIndex]);
38 }
39 );
40 // DELETE /attendance/:id - Delete an attendance record
41 app.delete('/attendance/:id', (req, res) => {
42   const recordId = parseInt(req.params.id);
43   const recordIndex = attendanceRecords.findIndex(record => record.id === recordId);
44   if (recordIndex === -1) {
45     return res.status(404).json({ error: 'Record not found' });
46   }
47   attendanceRecords.splice(recordIndex, 1);
48   res.json({ message: 'Record deleted successfully' });
49 });
50 // Start the server
51 const PORT = process.env.PORT || 3000;
52 app.listen(PORT, () => {
53   console.log(`Server is running on port ${PORT}`);
54 });
55
56
57
58
59
60
61

```

Output:-

```

Pretty-print ✓
[
  {
    "id": 1,
    "studentName": "John Doe",
    "date": "2023-10-01",
    "status": "present"
  },
  {
    "id": 2,
    "studentName": "Jane Smith",
    "date": "2023-10-01",
    "status": "absent"
  }
]

```

Observation:

This code implements a lightweight RESTful API for managing student attendance records using Node.js and Express. It provides endpoints to retrieve all records, create new entries, update existing ones, and delete records as needed. The application uses an in-memory array for data storage, making it ideal for testing, prototyping, and development environments.

The API includes basic validation and error handling to ensure required fields are present and that operations are performed only on valid records. All responses are structured in JSON format, ensuring consistency and seamless integration with front-end applications or other services.

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

- Endpoints required:
 - o GET /inventory → List all items
 - o POST /inventory → Add new item
 - o PUT /inventory/{id} → Update stock quantity
 - o DELETE /inventory/{id} → Remove an item
- Implement validation for stock values.

Expected Output:

- API capable of managing inventory stock levels.

Prompt:-

Create a simple RESTful API using Node.js and Express to manage warehouse inventory, storing items (ID, name, and stock) in an in-memory array.

Add routes to view all items, add new ones, update stock for a specific item, and delete items when needed.

Make sure the stock quantity can't go below zero—if it does, return a 400 error.

Keep responses clear and consistent in JSON format with proper status codes and helpful messages.

Code:-

```

JS assertion.js > app.put('/items/id') callback > item > inventory.find() callback
1 // Create a simple RESTful API using Node.js and Express to manage warehouse inventory, storing items (ID, name, and stock) in
2 // Add routes to view all items, add new ones, update stock for a specific item, and delete items when needed.
3 // Make sure the stock quantity can't go below zero—if it does, return a 400 error.
4 // Keep responses clear and consistent in JSON format with proper status codes and helpful messages.
5
6 const express = require('express');
7 const app = express();
8 app.use(express.json());
9
10 let inventory = [];
11 app.get('/items', (req, res) => {
12   res.json(inventory);
13 }
14 );
15 app.post('/items', (req, res) => {
16   const { id, name, stock } = req.body;
17   if (id === undefined || name === undefined || stock === undefined) {
18     return res.status(400).json({ message: 'ID, name, and stock are required.' });
19   }
20   if (stock < 0) {
21     return res.status(400).json({ message: 'Stock cannot be negative.' });
22   }
23   const newItem = { id, name, stock };
24   inventory.push(newItem);
25   res.status(201).json(newItem);
26 }
27 );
28 app.put('/items/:id', (req, res) => {
29   const { id } = req.params;
30   const { stock } = req.body;
31   const item = inventory.find(i => i.id === id);
32   if (!item) {
33     return res.status(404).json({ message: 'Item not found.' });
34   }
35   if (stock < 0) {
36     return res.status(400).json({ message: 'Stock cannot be negative.' });
37   }

```

```

JS assertion.js > app.put('/items/:id') callback > item > inventory.find() callback
15  app.post('/items', (req, res) => {
26  }
27  );
28  app.put('/items/:id', (req, res) => {
29      const { id } = req.params;
30      const { stock } = req.body;
31      const item = inventory.find(i => i.id === id);
32      if (!item) {
33          return res.status(404).json({ message: 'Item not found.' });
34      }
35      if (stock < 0) {
36          return res.status(400).json({ message: 'Stock cannot be negative.' });
37      }
38      item.stock = stock;
39      res.json(item);
40  });
41  app.delete('/items/:id', (req, res) => {
42      const { id } = req.params;
43      const index = inventory.findIndex(i => i.id === id);
44      if (index === -1) {
45          return res.status(404).json({ message: 'Item not found.' });
46      }
47      inventory.splice(index, 1);
48      res.json({ message: 'Item deleted successfully.' });
49  });
50  const PORT = 3000;
51  app.listen(PORT, () => {
52      console.log(`Server is running on port ${PORT}`);
53  });

```

Output:-

```

Pretty-print ☒
[
  {
    "id": 1,
    "name": "Item A",
    "stock": 10
  },
  {
    "id": 2,
    "name": "Item B",
    "stock": 20
  },
  {
    "id": 3,
    "name": "Item C",
    "stock": 30
  }
]

```

Justification:

The code implements a RESTful API for Warehouse Inventory Management using Node.js and Express. It provides endpoints for listing all items, adding new items, updating stock quantity, and removing items. The code includes validation checks to ensure that stock

quantity cannot be negative and returns appropriate error messages for invalid input or when an item is not found.

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

- Endpoints required:
 - o GET /rooms → View available rooms
 - o POST /rooms/book → Book a room
 - o GET /rooms/{id} → View booking details
 - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.

Prompt:-

Create a RESTful API for a Hotel Room Booking system using Node.js and Express that manages an array of rooms containing an ID, room number, and booking status. Develop a GET /rooms endpoint that returns only available rooms, a POST /rooms/book endpoint to book a specific room, a GET /rooms/:id endpoint to view specific booking details, and a DELETE /rooms/:id endpoint to cancel a booking. Implement error handling to return a message if a user tries to book a room that is already occupied or does not exist.

Code:-

```
assertion.js > ...
1 // Create a RESTful API for a Hotel Room Booking system using Node.js and Express that manages an array
2
3 const express = require('express');
4 const app = express();
5 app.use(express.json());
6
7 let rooms = [
8   { id: 1, roomNumber: '101', isBooked: false },
9   { id: 2, roomNumber: '102', isBooked: false },
10  { id: 3, roomNumber: '103', isBooked: false },
11  { id: 4, roomNumber: '104', isBooked: false },
12  { id: 5, roomNumber: '105', isBooked: false }
13 ];
14
15 // GET /rooms - Get available rooms
16 app.get('/rooms', (req, res) => {
17   const availableRooms = rooms.filter(room => !room.isBooked);
18   res.json(availableRooms);
19 });
20
21 // POST /rooms/book - Book a specific room
22 app.post('/rooms/book', (req, res) => {
23   const { id } = req.body;
24   const room = rooms.find(r => r.id === id);
25   if (!room) {
26     return res.status(404).json({ message: 'Room not found' });
27   }
28   if (room.isBooked) {
29     return res.status(400).json({ message: 'Room is already booked' });
30   }
31   room.isBooked = true;
32   res.json({ message: `Room ${room.roomNumber} booked successfully` });
33 }
34 );
35
36 // GET /rooms/:id - View specific booking details
```

```

JS assertion.js > ...
34 }
35 );
36 // GET /rooms/:id - View specific booking details
37 app.get('/rooms/:id', (req, res) => {
38   const { id } = req.params;
39   const room = rooms.find(r => r.id === parseInt(id));
40   if (!room) {
41     return res.status(404).json({ message: 'Room not found' });
42   }
43   res.json(room);
44 });
45 // DELETE /rooms/:id - Cancel a booking
46 app.delete('/rooms/:id', (req, res) => {
47   const { id } = req.params;
48   const room = rooms.find(r => r.id === parseInt(id));
49   if (!room) {
50     return res.status(404).json({ message: 'Room not found' });
51   }
52   if (!room.isBooked) {
53     return res.status(400).json({ message: 'Room is not booked' });
54   }
55   room.isBooked = false;
56   res.json({ message: `Booking for room ${room.roomNumber} cancelled successfully` });
57 });
58 const PORT = process.env.PORT || 3000;
59 app.listen(PORT, () => {
60   console.log(`Server is running on port ${PORT}`);
61 }
62 );
63 );
64

```

Output:-

```

Pretty-print ✓
[
  {
    "id": 1,
    "roomNumber": "101",
    "isBooked": false
  },
  {
    "id": 2,
    "roomNumber": "102",
    "isBooked": false
  },
  {
    "id": 3,
    "roomNumber": "103",
    "isBooked": false
  }
]

```

Observation:

This code implements a RESTful API for a Hotel Room Booking system using Node.js and Express. It provides endpoints to view available rooms, book a specific room, view booking

details, and cancel a booking. The API includes error handling to ensure that users cannot book already occupied rooms or cancel bookings for rooms that are not currently booked and returns appropriate JSON responses for each operation.

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system.

Instructions:

- **Endpoints required:**
 - o GET /candidates → List candidates
 - o POST /vote → Cast a vote
 - o GET /results → View voting results
 - Ensure one vote per user using in-memory validation.
- Expected Output:**
- API simulating a secure online voting process.

Prompt:-

Develop a RESTful API for an Online Voting System using Node.js and Express that manages a list of candidates and tracks user participation to ensure one vote per person. Include a GET /candidates endpoint to list all candidates, a POST /vote endpoint that accepts a userId and candidateId while checking an in-memory list to prevent duplicate voting, and a GET /results endpoint to display the current vote totals. If a user attempts to vote a second time, the API should return a message stating they have already voted.

Code:-

```
JS assertion.js •
1 // Develop a RESTful API for an Online Voting System using Node.js and Express that manages a list of candidates a
2
3 const express = require('express');
4 const app = express();
5
6 app.use(express.json());
7
8 // In-memory data storage
9 const candidates = [
10   { id: 1, name: 'Candidate A', votes: 0 },
11   { id: 2, name: 'Candidate B', votes: 0 },
12   { id: 3, name: 'Candidate C', votes: 0 }
13 ];
14 const voters = new Set();
15 // GET /candidates - List all candidates
16 app.get('/candidates', (req, res) => {
17   res.json(candidates);
18 });
19 // POST /vote - Cast a vote
20 app.post('/vote', (req, res) => {
21   const { userId, candidateId } = req.body;
22   if (voters.has(userId)) {
23     return res.status(400).json({ message: 'You have already voted.' });
24   }
25   const candidate = candidates.find(c => c.id === candidateId);
26   if (!candidate) {
27     return res.status(404).json({ message: 'Candidate not found.' });
28   }
29   candidate.votes += 1;
30   voters.add(userId);
31   res.json({ message: 'Vote cast successfully.' });
32 }
33 );
34 // GET /results - Display current vote totals
35 app.get('/results', (req, res) => {
36   const results = candidates.map(c => ({ name: c.name, votes: c.votes }));
37   res.json(results);
38 }
```

```
20 app.post('/vote', (req, res) => {
21   // Check if user has already voted
22   return res.status(400).json({ message: 'You have already voted.' });
23 }
24
25 const candidate = candidates.find(c => c.id === candidateId);
26 if (!candidate) {
27   return res.status(404).json({ message: 'Candidate not found.' });
28 }
29 candidate.votes += 1;
30 voters.add(userId);
31 res.json({ message: 'Vote cast successfully.' });
32 }
33 );
34 // GET /results - Display current vote totals
35 app.get('/results', (req, res) => {
36   const results = candidates.map(c => ({ name: c.name, votes: c.votes }));
37   res.json(results);
38 });
39 // Start the server
40 const PORT = process.env.PORT || 3000;
41 app.listen(PORT, () => {
42   console.log(`Server is running on port ${PORT}`);
43 });
44
45
```

Output:-

```
Pretty-print ✓
[
  {
    "id": 1,
    "name": "Candidate A",
    "votes": 0
  },
  {
    "id": 2,
    "name": "Candidate B",
    "votes": 0
  },
  {
    "id": 3,
    "name": "Candidate C",
    "votes": 0
  }
]
```

Justification:

This code implements a RESTful API for an Online Voting System using Node.js and Express. It provides endpoints to list candidates, cast votes while ensuring one vote per user, and display current vote totals. The API includes error handling to prevent duplicate voting and returns appropriate JSON responses for each operation.

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

- **Endpoints required:**
 - o GET /teachers → List all teachers
 - o POST /teachers → Add a new teacher
 - o PUT /teachers/{id} → Update teacher details
 - o DELETE /teachers/{id} → Remove a teacher
- Ensure proper error handling and JSON responses.

Expected Output:

- **Functional API for managing teacher information.**

Prompt:-

Build a RESTful API for Teacher Management using Node.js and Express that allows for full CRUD operations on teacher records consisting of ID, name, subject, and department.

Implement a GET /teachers endpoint to view the list, a POST /teachers endpoint to add a new record, a PUT /teachers/:id endpoint to update teacher details, and a DELETE /teachers/:id endpoint to remove a record. Ensure the API returns proper JSON responses and 404 error messages if a requested teacher ID is not found in the system.

Code:

```
//Build a RESTful API for Teacher Management using Node.js and Express that allows for full CRUD
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let teachers = [
  // Sample teachers for testing
  { id: 1, name: 'John Doe', subject: 'Mathematics', department: 'Science' },
  { id: 2, name: 'Jane Smith', subject: 'English', department: 'Arts' },
  { id: 3, name: 'Emily Johnson', subject: 'History', department: 'Social Studies' }
];
let nextId = 4;
// GET /teachers - List all teachers
app.get('/teachers', (req, res) => {
  res.json(teachers);
});
// POST /teachers - Add new teacher
app.post('/teachers', (req, res) => {
  const { name, subject, department } = req.body;
  if (!name || !subject || !department) {
    return res.status(400).json({ error: 'Name, subject, and department are required.' });
  }
  const newTeacher = { id: nextId++, name, subject, department };
  teachers.push(newTeacher);
  res.status(201).json(newTeacher);
});
// PUT /teachers/:id - Update teacher details
app.put('/teachers/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const { name, subject, department } = req.body;
  const teacher = teachers.find(t => t.id === id);
  if (!teacher) {
    return res.status(404).json({ error: 'Teacher not found.' });
  }
  if (name) teacher.name = name;
  if (subject) teacher.subject = subject;
  if (department) teacher.department = department;
  res.json(teacher);
});
// DELETE /teachers/:id - Remove a teacher
```

```
});  
// DELETE /teachers/:id - Remove a teacher  
app.delete('/teachers/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const index = teachers.findIndex(t => t.id === id);  
  if (index === -1) {  
    return res.status(404).json({ error: 'Teacher not found.' });  
  }  
  teachers.splice(index, 1);  
  res.json({ message: 'Teacher removed successfully.' });  
});  
// Start the server  
app.listen(port, () => {  
  console.log(`Server is running on http://localhost:\${port}`);  
});
```

OUTPUT:

```
Pretty-print ✓  
[  
  {  
    "id": 1,  
    "name": "John Doe",  
    "subject": "Mathematics",  
    "department": "Science"  
  },  
  {  
    "id": 2,  
    "name": "Jane Smith",  
    "subject": "English",  
    "department": "Arts"  
  },  
  {  
    "id": 3,  
    "name": "Emily Johnson",  
    "subject": "History",  
    "department": "Social Studies"  
  }  
]
```

Justification:

This code implements a RESTful API for Teacher Management using Node.js and Express. It provides endpoints to view the list of teachers, add new teacher records, update existing teacher details, and remove teacher records. The API includes error handling to ensure that required fields are provided when adding or updating a teacher and returns appropriate JSON responses for each operation, including 404 errors when a requested teacher ID is not found in the system.

